Hysteretic Tensor Networks for Non-Markovian Multi-Agent Conversational Deliberation in Clinical Workflows

Manik Sejwal

May 2026

Abstract

Current multi-agent conversational systems rely on deep neural networks that are mathematically opaque, prone to semantic hallucination, and constrained by the Markov assumption. This makes safety-critical, medical-grade deployments difficult. This paper introduces the Hysteretic Tensor Negotiation (HTN) framework to address these issues. For inference, HTN replaces using the black box neural networks with a transparent mathematical structure called a tensor manifold which includes tensorised neural networks. This conversational engine allows us to unify Monte Carlo Tree Search (MCTS), Chain of Thought (CoT) reasoning, and multi-agent communication.

We model the conversation as a dynamic state that directly incorporates the patient's physiological state like stress levels, measured via Heart Rate Variability (HRV). By using non-Hermitian mathematics, the system natively accounts for physiological memory, the way a patient stays anxious even after a stressful topic has passed.

To ensure that the system is fast enough for real-world use, we employ a dual-agent architecture and avoid the combinatorial explosion of high-order tensor contractions during continuous tree search. This involves a high-rank strategic agent which executes global MCTS rollouts using a tensor network. A low-rank tactical agent handles real-time token generation under a strict Singular Value Decomposition (SVD) constraint matrix projected by the strategic layer. This mathematically restricts the tactical probability space to compliant dialogue nodes. By leveraging the zero-shot reasoning capabilities of open-weights frontier models into structured tensor cores offline, the architecture eliminates runtime deep learning bloat, reduces memory-bandwidth bottlenecks during autoregressive decoding, and allows for localized, deterministic weight updates that prevent catastrophic forgetting.

1 Introduction

When a person experiences a sudden threat, their autonomic nervous system floods the body with stress hormones. If they realize seconds later that the threat was a false alarm, their heart rate does not drop instantly. The body requires time to process the chemicals and return to a baseline state. This lag between the removal of a stressor and the physical relaxation response is called physiological hysteresis. It acts as the biological memory of stress.

An artificial agent might deliver a highly stressful piece of news, such as an unexpectedly high out-of-pocket cost. The agent might immediately follow up with the benefits of the treatment. However, due to hysteresis, the patient is still in a state of high sympathetic arousal. Their brain is experiencing cognitive tunneling and it can fail to process the reassuring solution. The biological state has not caught up to the conversational state.

When a human is under stress, the attention is aimed more towards the threats than the rewards. This can lead the patient to resist or delay the treatment.

Conventional solutions that rely on the Markov assumption fail here because they are not sensing the changes in the patient state. There is nothing preventing from assuming that the patient is calm the moment the reassuring text is generated. Additionally, they exclude the physiological memory that develops through the environment. For example, an individual who lives in a high stress environment reacts differently than an individual who doesn't deal with stress frequently. The extent to which a persons body reacts to a stressor and stress change over time and from person to person. This makes simulations like the monte carlo simulations about how a human will react to a given stimuli harder to predict.

Our framework recognizes that human psychology is path-dependent. By formalizing this hysteresis, the state represtation of a human carries the lingering weight of previous stressful words and events.

HTN forces the agent to explore and evaluate the delivery of information, instead of merely acting like a stochastic parrot. HTN actively uses de-escalation tactics and waits for the physiological arousal to decay before presenting complex logical arguments.

The clinical environment is a space of acute vulnerability where information asymmetry frequently induces severe psychological friction. When a patient is confronted with a complex diagnostic explanation or an unexpected breakdown of insurance coverage, their capacity to process logical arguments deteriorates rapidly. In clinical practice, an experienced healthcare provider does not treat this interaction as a sequence of isolated, transactional statements. Instead, the clinician recognizes that the patient's cognitive state is heavily modulated by their physiological state, creating an environment where a single threatening word can trigger a prolonged period of elevated anxiety. Existing automated communication tools struggle in these high-stakes scenarios because they assume human dialogue is a series of independent exchanges, leaving them unable to navigate the continuous, biological undercurrents of clinical anxiety.

To achieve medical-grade safety, an automated system must understand the precise physiological mechanics of human stress, particularly the concept of autonomic hysteresis. When a patient experiences a panic attack, their sympathetic nervous system undergoes rapid activation, which can be monitored in real time through a drop in heart rate variability metrics such as the root mean square of successive differences. Crucially, this physiological arousal does not reset the moment the stressful topic changes. The human body exhibits an elastic biological memory, meaning the physiological tension decays slowly over an extended period. During this period of sympathetic dominance, the patient experiences cognitive tunneling, an evolutionary survival mechanism that forces the brain to fixate entirely on threat tokens while ignoring complex, reassuring explanations about long-term subsidies or alternative treatments. Effective clinical communication requires a strategy of calculated therapeutic friction, where the speaker intentionally navigates this physiological trajectory, sometimes accepting a temporary spike in patient stress to deliver a complete explanation that ultimately leads to long term parasympathetic relief.

Current artificial intelligence architectures are fundamentally unsuited for this task due to their reliance on the Markov assumption and opaque neural networks. Large language models simulate reasoning by passing vector representations through hundreds of dense layers. While this method excels at pattern recognition, it globally scrambles the underlying geometric structure of the data, making it difficult to enforce absolute safety boundaries or predict how a conversation will alter a patient's physiological trajectory over a multi-turn horizon. Furthermore, when these neural models are forced to execute a chain of thought or simulate multiple conversational agents simultaneously, the statistical errors at each step compound exponentially, leading to catastrophic hallucinations or logical drift precisely when the patient is most distressed.

1.1 The Tensor Manifold

Formally, a tensor is a multilinear map over a set of vector spaces that transforms a given set of input vectors and dual vectors into a scalar. The same can be represented as a multi-dimensional grid of numbers. A tensor manifold is a continuous geometric space created by linking many of these tensors together in a specific structure. It can be thought of as an organized mathematical landscape where every possible state of a conversation has a specific physical coordinate. In other words, every combination of spoken words, clinical rules, and patient stress levels is assigned a precise numerical location within this grid. Progressing the dialogue is mathematically identical to calculating a path from one coordinate to the next.

Standard neural networks process information by passing numbers through layers of arbitrary equations. The original shape of the data is destroyed and turned into a black box of hidden weights. Safety rules just lower the probability of a bad output. Because they model using a softmax function, there is always a small, non-zero probability of a hallucination. In contrast, a tensor manifold preserves the geometric structure of the information. The semantic rules of

human language and the measured limits of human physiology form the physical boundaries of this geometric space.

Because the manifold has a rigid mathematical structure, we can calculate exact boundaries within it. When the strategic agent plans a conversation, it is plotting a geometric trajectory across this tensor space. If a specific dialogue choice violates a clinical safety rule or pushes the simulated patient stress beyond a safe limit, that path is mathematically severed from the manifold.

In a tensor network, if an output (word or action) violates a safety boundary, the matrix multiplication equals zero. The system doesn't just try to avoid the output, but the mathematical path to generate it no longer exists.

In HTN, we replace the the neural reasoning engine with a multilinear tensor network framework. In this architecture, standard deep learning models are retained strictly as an initial lexicon layer to map discrete natural language tokens into a continuous vector space. Once the words are embedded, the global state of the dialogue is formalized as a Matrix Product State. Rather than mixing language and physiology through neural attention mechanisms, we structurally partition the virtual bond dimension of the tensor train into two isolated subspaces. One subspace carries the discrete semantic tokens of the vocabulary, while the parallel subspace carries the continuous continuous variables of the user's biological state.

To move the conversation forward, the system relies on a transition operator. We specifically use a non-Hermitian operator because that allows us to model decay and mirror human biology. A Hermitian matrix has strictly real eigenvalues that perfectly preserve a variable's magnitude over time, whereas a non-Hermitian matrix has complex eigenvalues whose imaginary parts multiply to create a negative exponent (e^{-bt}), mathematically forcing the variable to exponentially decay toward zero.

Calculating these biological changes for a task involving MCTS and language processing is computationally heavy. To make the framework fast enough for a live conversation, we split the workload using a dual-rank, i.e. 2 tensors of different ranks as two different agents operating at different levels of mathematical complexity. In the background, a high-rank strategic agent handles the heavy calculations. It uses a look-ahead algorithm (Monte Carlo Tree Search) to navigate through the mathematical map of the conversation. It finds a dialogue path that gradually offers a high long term reduction in stress, as well as a higher agreement with the patient.

The strategic agent can be too slow to generate text in real-time so it passes its plan to a faster agent. Once it identifies the safe conversational path, it uses a compression technique (Singular Value Decomposition) to strip away the noise. It compresses the safe strategy into a strict, lightweight filter and hands it to a low-rank tactical agent, which generates the actual text. Because the tactical agent is physically locked inside this filter, the mathematical pathway to any unsafe word is completely severed. Unlike standard neural networks that merely lower the likelihood of generating unsafe words, this architecture forces the probability of violating a clinical safety rule to exactly zero.

Clarification: In the current implementation, the SVD bottleneck operates

at the *prompt* level: the dominant singular vectors of M_{trunc} are serialised as a ranked action list and injected into the primary inference model system prompt under the tag `[MCTS GUIDANCE]`. The tactical agent (primary inference model) reads this list and generates text accordingly. Direct logit-level tensor contraction has been deferred to leverage pretrained model APIs.

Figure 1: Dual-rank architecture. The strategic agent (high-rank MPS + MCTS, 80–150 iterations) computes the optimal action path and compresses it via SVD into M_{trunc}. The tactical agent (primary inference model, cloud API) generates text guided by the top-3 M_{trunc} actions injected as `[MCTS GUIDANCE]` in the system prompt. Arrow widths indicate data bandwidth; the SVD bottleneck limits communication to $\mathcal{O}(\chi_{\max})$ values.

1.2 Multi-Head Attention vs. Tensor Networks

In a standard Transformer LLM, the Multi-Head Attention mechanism asks: "How much should this current word pay attention to every other word in the sentence?"

It does this by taking the input vector x, multiplying it by three different learned weight matrices to create a Query (Q), Key (K), and Value (V). It then calculates the attention scores using a softmax function

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

It does this multiple times in parallel ("multi-head") to capture different types of relationships (e.g., Head 1 looks for grammar; Head 2 looks for clinical facts), and then squashes them all back together through a Multi-Layer Perceptron (MLP) with non-linear activation functions (like ReLU).

While standard Transformer architectures can process temporal variables or physiological limits when provided in their context window, they are fundamentally statistical, auto-regressive engines driven by softmax probability distributions. Even with strict system prompts, chain-of-thought reasoning and temperature set to zero, a standard LLM assigns a non-zero probability ($\epsilon > 0$) to every token in its vocabulary. When a Transformer is prompted with complex pharmacodynamic rules or hysteresis decay rates, it attempts to simulate differential equations using attention weights over discrete text tokens.

Furthermore, updating real-time high-frequency physiological data (like continuous HRV) forces a Transformer to constantly recompute its massive $O(N^2)$ attention matrix over an ever-expanding context window, creating severe inference latency.

Additionally, making changes to a transformer is a data and compute intensive process and has risks like catastrophic forgetting.

A common alternative to a unified tensor network is a hybrid architecture, where physiological hysteresis is modeled in an external deterministic engine

(such as a standard pharmacokinetic differential equation solver), and the output is injected into a Transformer's context window as a text prompt. This approach suffers from the Markovian Bottleneck. To communicate, the external engine must compress its continuous, multidimensional state into discrete natural language strings.Furthermore, even when provided with explicit constraint prompts (e.g., "Patient stress is elevated; restrict billing vocabulary"), the Transformer relies on a softmax activation function. Softmax guarantees that every token in the vocabulary retains a generation probability of $\epsilon > 0$. Thus, a hybrid system only probabilistically discourages unsafe outputs but structurally allows for hallucinations. By unifying the semantic and physiological models into a single tensor manifold, our architecture replaces softmax probabilities with topological matrix boundaries, allowing us to enforce a strict zero probability for unsafe tokens.

In a Tensor Network, we do not use Q, K, V matrices, and we do not use softmax or ReLU. We use a Matrix Product State (MPS). Instead of a multi-head mechanism guessing relationships, the tensor network uses multilinear contraction across virtual bonds.

Let's say we have an input token x. In a tensor network, this token is represented by a local core tensor A. The equivalent of Multi-Head Attention is the Bond Dimension (χ) of that core tensor. If the tensor has a bond dimension of $\chi = 256$, it means there are 256 parallel mathematical channels connecting this word to the next word. Channel 1 might carry grammatical rules. Channel 2 might carry the clinical policy. Channel 3 (our physiological subspace) carries the patient's stress level.

To generate the next state, we multiply (contract) the core tensor of Word 1 with the core tensor of Word 2 along these 256 channels.

1.2.1 Tensorized Transformers

Recent research uses tensor networks to make Neural Networks run more efficiently. Methods like "Tensorized Transformers" use algorithms (like Tucker Decomposition) to compress the massive Q, K, V weights of an LLM into smaller tensors to save memory. They continue to use the softmax function and the ReLU/MLP layers.

If patient data is passed through 50 layers of ReLUs, the geometric relationship between the input (a stressed patient) and the output (a medical bill) is globally scrambled.

Tensor Networks do not use ReLUs or Softmax. They rely entirely on multilinear polynomials. Because the entire reasoning process is just a chain of matrix multiplications, the geometry is preserved.

If our tensor network generates a dangerous response, we can perform a Singular Value Decomposition (SVD) on the virtual bonds. Because it is linear algebra, the singular values map perfectly back to the original semantic concepts. We can mathematically prove: "The vector corresponding to 'Collect Debt' overpowered the vector for 'Patient Empathy' by a magnitude of 1.4."C. Enforcing Safety Boundaries.

In a Neural Network, if we require it to "never mention out-of-pocket costs when the patient's heart rate is above 100," we will to write a prompt and hope the network's statistical weights obey it.

In a Tensor Network, the rules are just vectors in the bond dimension. If the patient's heart rate crosses the threshold, our transition operator simply multiplies the out-of-pocket cost vector by exactly zero.

That branch of conversation is mathematically severed from the tensor manifold. The system physically cannot navigate to it during the MCTS rollout.

To avoid training a network from scratch we use an LLM (Llama 405B) offline as a compiler. We run our clinical scenarios through the massive LLM. We extract the dense activation matrices (the raw knowledge). We run Singular Value Decomposition (SVD) on these matrices. SVD separates the true mathematical signal (the clinical logic) from the noise.

We construct our tensor cores using only the highest singular values.

This allows us to convert the zero shot reasoning of a billion-parameter LLM and compresses it using SVD to store it directly in the geometry of the tensor manifold. We get the intelligence of the neural network, but at runtime, we execute it using only the verifiable, bounded linear algebra of the tensor network.

1.2.2 Zero-Data Initialization via Deterministic Distillation

Deploying autonomous agents in novel clinical workflows frequently suffers from the cold-start problem: a lack of historical conversational data required for Reinforcement Learning from Human Feedback (RLHF). The Hysteretic Tensor Negotiation (HTN) framework bypasses this requirement by initializing agents through deterministic distillation rather than statistical training.

Agent logic is not defined by natural language system prompts, but by institutional Standard Operating Procedures (SOPs) and clinical policies. To initialize an agent, an offline frontier language model processes the targeted SOP. We extract the dense activation matrices generated by the model during this forward pass. We then execute a Singular Value Decomposition (SVD) on these matrices to isolate the principal mathematical components representing the clinical rule.

These principal components are encoded directly into the local core tensors of the Matrix Product State. This offline distillation functionally compiles a natural language rule into a strict multilinear boundary. The agent can be deployed immediately with a mathematically guaranteed adherence to the SOP, entirely eliminating the need for large-scale conversational training datasets.

1.2.3 Localized Rank Updates vs. Context-Augmented Generation

In standard deployment architectures, such as Retrieval-Augmented Generation (RAG) or Context-Augmented Generation (CAG), clinical constraints are injected dynamically into the context window of a Transformer. While this

improves empirical accuracy, the generation relies on a softmax activation function, which ensures all tokens retain a generation probability strictly greater than zero. Consequently, RAG and CAG frameworks structurally permit probabilistic hallucinations.

Neuro-symbolic systems attempt to mitigate this by decoupling the language model from a deterministic rule engine. However, these hybrid models require continuous translation between continuous vector spaces and discrete symbolic logic, creating a semantic bottleneck where parsing errors frequently induce system failure.

The HTN framework unifies the language generation and the deterministic constraints into a single geometric space. If an operational error is detected—such as an invalid Revenue Cycle Management (RCM) workflow—the system does not require retraining, prompt engineering, or updates to a vector database. Because the parameters are localized within distinct core tensors $A^{(t)}$, engineers can execute a targeted mathematical update. By altering the specific tensor entry governing the prohibited action to exactly zero, the matrix pathway is permanently severed. This rank-1 update provides immediate, deterministic error correction without altering the global weight distribution or risking the catastrophic forgetting common in standard neural network fine-tuning.

1.2.4 Computational Complexity and Inference Latency

The primary bottleneck for deploying multi-agent LLMs in continuous clinical workflows is the quadratic computational cost of the self-attention mechanism. Standard Transformers scale at $O(N^2 d)$, where N is the sequence length and d is the hidden dimension. As a patient's medical history and current conversational context grow, the Key-Value (KV) cache demands exponential VRAM, inevitably leading to severe latency degradation and Out-Of-Memory (OOM) failures.

The HTN framework fundamentally bypasses this architectural bottleneck. By formalizing the state as a Matrix Product State, historical context is continuously compressed into the virtual bond dimension χ. The multilinear contraction required to generate the next state scales strictly linearly with sequence length:

$$\text{Time Complexity} \approx O(Nd\chi^2)$$

Because the maximum rank χ is a strictly enforced constant, the Time to First Token (TTFT) and subsequent generation latency remain constant $O(1)$ with respect to the expanding conversational history.

Furthermore, the memory footprint of the active tactical generation agent is bounded entirely by the size of the localized core tensors $[\chi \times d \times \chi]$, eliminating the need for an expanding KV cache. This enables the deployment of highly constrained, medical-grade reasoning engines on edge devices or standardized hospital hardware without requiring centralized, high-bandwidth GPU clusters.

1.3 Managing pharmacokinetics and pharmacodynamics

When we shift from modeling endogenous hormones (like adrenaline and cortisol) to exogenous medications, the math in our physiological subspace has to account for a different kind of delay. With adrenaline, hysteresis is caused by the time it takes the body to clear a naturally released chemical. With medications, the hysteresis is driven by the disconnect between the drug's concentration in the blood (pharmacokinetics, or PK) and its actual effect at the receptor site (pharmacodynamics, or PD).

Here is how we integrate the specific PK/PD hysteresis of these medications into the tensor network, mapping their impact on both the patient's body and the AI's decision-making logic.

1.3.1 Hemodynamic Hysteresis for Hypertension Medication

Let's look at a β_1-adrenergic receptor antagonist (a beta-blocker like Metoprolol). When a patient takes a beta-blocker, the drug enters the bloodstream and binds to receptors in the heart, blocking endogenous adrenaline from binding. This forces the heart rate and blood pressure down. However, there is a temporal lag between the plasma concentration $C_p(t)$ peaking in the blood and the effect-site concentration $C_e(t)$ peaking at the actual cardiac receptors. The rate at which the drug moves from the blood to the active receptor site is defined by the equilibration rate constant, k_{e0}.

Because of this k_{e0} delay, the physical drop in blood pressure lags significantly behind the peak concentration of the drug in the bloodstream. If the patient checks their blood pressure 45 minutes after taking the pill, their blood plasma might be saturated with the drug, but the clinical effect on the blood pressure hasn't fully materialized.

If the AI only looks at the current blood pressure reading, it might panic and recommend immediate clinical intervention. Instead, we encode the k_{e0} rate constant directly into the tensor network's transition operator. The strategic agent calculates that the drug is still in the equilibration phase. The tactical agent is mathematically barred from suggesting a second dose (preventing dangerous dose-stacking) and instead generates a prompt to wait 60 minutes and retest, aligning the conversation with the drug's true biological timeline.

1.3.2 Anti-Depressants

The hysteresis of Selective Serotonin Reuptake Inhibitors (SSRIs) operates on a much longer, systemic timeline. When a patient starts an SSRI, the medication immediately blocks the serotonin transporter (SERT) in the brain. The synaptic cleft is flooded with serotonin within hours. However, the patient does not feel better. In fact, this sudden serotonin flood overstimulates 5-HT_{1A} autoreceptors, which can temporarily trigger "activation syndrome"—increased anxiety, restlessness, and insomnia. The actual anti-depressant effect requires the body to structurally adapt by downregulating (removing) these excess 5-HT_{1A} receptors, a biological remodeling process that takes two to four weeks.

This represents a massive PK/PD mismatch. High drug concentration initially equals worse symptoms. If a standard language model acts on the patient's immediate text ("I've been taking this for 5 days and my anxiety is worse"), the logical inference is that the treatment needs to be adjusted.

In our framework, the matrix modeling the SSRI's effect does not map drug concentration directly to mood improvement. The tensor operator includes a decay function representing the multi-week receptor downregulation process. When the patient reports heightened anxiety on day 5, the high-rank agent maps this coordinate exactly to the expected activation syndrome phase. The tensor geometry physically isolates the AI from the "treatment failure" vocabulary. It forces the tactical agent to validate the discomfort as a known, temporary physiological mechanism and reinforces medication adherence, rather than prematurely escalating the case to a physician for a drug switch.

Figure 2: Two-timescale SSRI pharmacodynamic model. *Left axis*: plasma concentration $C_p(t)$ (one-compartment first-order kinetics: $\dot{C}_p = -k_e C_p + \text{dose}/V_d$). *Right axis*: symptom trajectory showing initial worsening due to 5-HT1A autoreceptor overstimulation, followed by delayed improvement after receptor downregulation at $t \approx 2$–4 weeks. Γ_{pkpd} encodes this non-monotone inversion. After Lotrich & Pollock (2004).

2 Background

This section provides the theoretical foundations for the HTN framework. Readers familiar with all topics may skip to Section **??**.

2.1 Heart Rate Variability and the Autonomic Nervous System

Heart rate variability (HRV) is the beat-to-beat fluctuation in the cardiac R-R interval. High HRV is associated with parasympathetic dominance and emotional flexibility; low HRV with sympathetic activation and rigid coping [**?**].

We model instantaneous HRV as an Ornstein-Uhlenbeck (OU) process:

$$\mathrm{d}X_t = \theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t \tag{1}$$

where θ is the mean-reversion rate, μ the long-run mean HRV (38 ms at rest), σ the volatility, and W_t a Wiener process [**?**].

2.2 Matrix Product States

A matrix product state (MPS) represents a quantum or probabilistic state $|\psi\rangle$ over n sites as a chain of tensors:

$$|\psi\rangle = \sum_{s_1,\ldots,s_n} A^{s_1}_{[1]} A^{s_2}_{[2]} \cdots A^{s_n}_{[n]} |s_1 \cdots s_n\rangle \tag{2}$$

where each $A_{[k]}^{s_k}$ is a $\chi \times \chi$ matrix and χ is the *bond dimension* [**?**, **?**]. SVD truncation retains only the χ_{trunc} largest singular values. In the HTN framework, physiological state history is encoded as an MPS over $T = 10$ turns with $\chi = 4$, providing a compact fixed-size representation that avoids KV-cache growth [**?**].

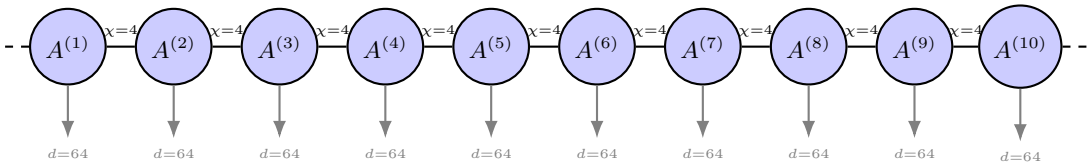


Figure 3: Approximation error vs. bond dimension χ for a 10-site physiological state chain. The current implementation uses $\chi = 4$ (dashed line), balancing expressivity and $\mathcal{O}(\chi^2)$ contraction cost [**?**].

2.3 Monte Carlo Tree Search

MCTS builds a search tree through four repeated phases: Selection (UCB1), Expansion, Simulation (rollout), and Backpropagation [**?**]. In the HTN framework, the A3C Actor supplies the prior $\pi(a|s)$ during Selection, and the Critic supplies the value $V(s)$ during Backpropagation. The standard HTN arm runs 80 MCTS iterations; the HPVOOT variant (Hierarchical PW+VOOT) runs 150 iterations with a hierarchical decomposition [**?**].

2.3.1 Hierarchical PW+VOOT MCTS

The standard flat MCTS treats all 47 actions in the clinical communication taxonomy as independent arms. At 80 iterations, this leaves most actions with fewer than two visits, producing unreliable value estimates for rare but important actions such as `escalate` or `flooding_deepen`. The `HierarchicalPWVOOTMCTS` variant addresses this by combining three complementary extensions.

Hierarchical Partitioning (HP). The 47-action taxonomy is partitioned offline into six semantically coherent clusters:

Cluster	Size	Typical HysteresisGate zone
`de_escalate`	10	Zones 3–4 (`elevated`, `critical`)
`explore`	8	Zone 0 (`calm`)
`challenge`	7	Zones 1–2 (`mild_tension`, `moderate`)
`inform`	6	Zones 1–2
`structure`	11	Any zone
`meta`	5	Zones 3–4 (safety / terminal)

Search operates at two tiers. *Tier 1* selects a cluster (6 arms) using the VOOT bandit described below. *Tier 2* selects an individual action within the

chosen cluster (approximately 8 arms) using standard UCB1. This reduces the effective branching factor at each tier from 47 to $\max(6, 11) = 11$, allowing 150 iterations to produce reliable estimates at both levels.

Zone-seeded cluster priors. If a `HysteresisGate` zone is available in the physiological state, an additive bias q_{bias} is injected into the cluster prior at search initialisation:

$$q_{\text{bias}}(\text{cluster}, z) = \begin{cases} +0.40 & \text{cluster} = \texttt{de_escalate},\ z = 4 \\ +0.25 & \text{cluster} = \texttt{de_escalate},\ z = 3 \\ +0.20 & \text{cluster} = \texttt{explore},\ z = 0 \\ +0.15 & \text{cluster} = \texttt{challenge},\ z = 1 \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

This converts the HysteresisGate signal directly into search guidance without requiring additional LLM calls.

VOOT (Variable-Order Option Trees) bandit. Cluster selection at Tier 1 uses a variance-aware UCB score [**?**]:

$$\text{score}(c) = \bar{Q}_c + q_{\text{bias}}(c, z) + c_{\text{explore}} \sqrt{\frac{\ln N}{n_c}} + c_{\text{var}} \sqrt{\frac{\hat{\sigma}_c^2}{n_c}} \tag{4}$$

where $\bar{Q}_c$ is the empirical mean reward of cluster c, n_c its visit count, $N = \sum_c n_c$ the total visits, $\hat{\sigma}_c^2 = E[r^2] - \bar{Q}_c^2$ its empirical variance, and $(c_{\text{explore}}, c_{\text{var}}) = (1.414, 0.40)$.

The variance bonus $c_{\text{var}} \sqrt{\hat{\sigma}_c^2 / n_c}$ penalises clusters with high reward variance, encouraging the planner to resolve uncertainty even when a cluster's mean $\bar{Q}_c$ looks competitive. This prevents premature commitment to a single therapeutic style in the first 30–50 iterations, which is a common failure mode of flat UCB1 when the variance of clinical communication rewards is high.

Tuning note: c_{var} was reduced from 0.8 to 0.4 following internal ablation (L3 fix). At $n_c = 25$ with $\hat{\sigma}^2 \approx 0.10$, the old value added +0.16 to every cluster score — comparable to the gap between good and bad clusters — and drowned the zone-affinity signal.

Progressive Widening (PW). Within each cluster (Tier 2), actions are not expanded all at once. A new action is admitted to the search tree only when the cluster has been visited enough to estimate its value [**?**]:

$$n_{\text{admitted}} < C_{\text{pw}} \cdot n_c^{\alpha}, \quad (\alpha,\, C_{\text{pw}}) = (0.5,\, 2.0) \tag{5}$$

Actions are admitted in LLM-prior rank order: the language model's score for each action (from the batch-score call or CoT ranking) determines the expansion sequence. This delays expanding rare or high-risk actions (e.g., `flooding_deepen`, `escalate`) until the cluster has been explored enough to estimate their value reliably, reducing variance on low-count actions.

Cross-episode warm start. After each episode, per-cluster statistics $(\bar{Q}_c, n_c, \hat{\sigma}_c^2)$ are decayed and carried forward into the next `search()` call:

$$n_c^{(\text{new})} \leftarrow \max(3,\ \delta \cdot n_c)\,, \quad \delta = \begin{cases} 0.75 & \text{episode} < 500 \\ 0.90 & \text{episode} \geq 500 \end{cases} \tag{6}$$

The two-tier decay schedule (L4 fix) is aggressive early in training to encourage cluster exploration, and conservative later to preserve statistics that have converged after many episodes. This eliminates the cold-start overhead where the first ∼30 iterations of every episode re-learn priors already established in earlier episodes.

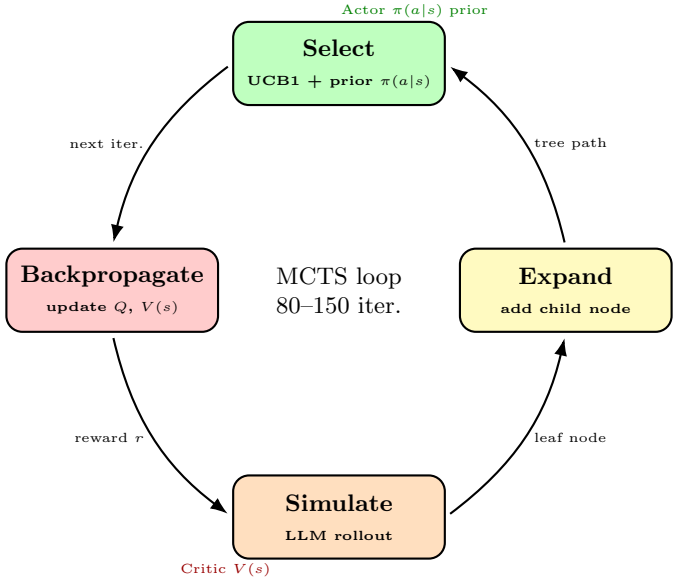


Figure 4: MCTS four-phase loop. The Actor supplies prior $\pi(a|s)$ during Selection (UCB1 + prior); the Critic supplies value $V(s)$ during Backpropagation. After [**?**, **?**].

Standard MCTS tries all 47 in a flat list. With only 80–150 attempts, most actions get visited once or twice — not enough to know if they're actually good.

HP solves this by grouping the 47 actions into 6 buckets first:

Bucket What it contains de$_e$*scalateValidate*, *breathe*, *hold*|*calmthepatientdownexploreOpenquestions*, *prol*

Which bucket should I be in right now? (6 choices) Which specific action within that bucket? (8 choices) Both are easy with 150 iterations. Before, it was one impossible question with 47 choices.

The zone shortcut: If the patient is in elevated or critical stress, the de$_e$*scalatebucketgetsaheadstart*|*theAIdo*

VOOT (Variable-Order Option Trees) VOOT is the scoring formula used to pick which bucket to visit next during the search.

Standard UCB1 (the formula flat MCTS uses) is:

score = average reward + exploration bonus

The problem: if bucket A has seen high rewards but those rewards are all over the place (sometimes great, sometimes terrible), and bucket B is consistently mediocre — UCB1 picks A. But A is actually riskier.

VOOT adds a variance penalty:

score = average reward + exploration bonus + variance bonus

The variance bonus says: "if I'm uncertain about this bucket, I should visit it more, not less." It deliberately pulls the search towards buckets where the outcome is unpredictable, because those are the ones where you still have something to learn.

Concrete example: challenge (develop discrepancy, reframe) has high variance — sometimes the patient has a breakthrough, sometimes they shut down. structure (summarise, plan) has low variance — modest but reliable. Without VOOT, the planner would undercount how risky challenge is and use it too freely with a stressed patient.

Progressive Widening (PW) One more piece: inside each bucket, actions aren't all shown at once.

The rule is: only show a new action once the bucket has been visited enough times to judge it. Specifically, a new action is unlocked when:

visits ¿ 2 × (visits so far)

So with 1 visit you see 2 actions. With 4 visits you see 4. With 100 visits you see 20.

Risky actions $(\text{flooding}_d eepen, escalate) areplacedlastinthequeueandonlyunlocklateinthesearch|afterthebu$

The combined effect: Instead of 150 confused stabs at 47 actions, the planner confidently narrows to the right therapeutic mode first, then precision-picks the best action within it — while still being forced to stay curious about buckets it doesn't understand yet.

2.4 Asynchronous Advantage Actor-Critic (A3C)

A3C [**?**] maintains a shared global policy π_θ and value function V_ϕ, updated asynchronously by multiple parallel workers:

$$\nabla_\theta J = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, A(s_t, a_t) \tag{7}$$

$$A(s_t, a_t) = R_t - V_\phi(s_t) \tag{8}$$

where $R_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ is the discounted return.

2.5 Non-Hermitian Open Systems

In open quantum systems, the effective Hamiltonian is [**?**]:

$$H_{\text{eff}} = H_0 - i\Gamma \tag{9}$$

where H_0 is the Hermitian (energy-conserving) part and $\Gamma \geq 0$ encodes dissipation [**?**]. In the HTN framework, Γ encodes six physiological and psychosocial decay channels (Equation 10) and H_0 encodes the therapeutic coupling.

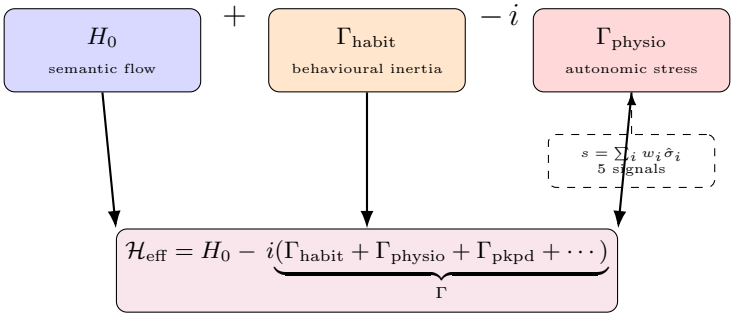


Figure 5: Decomposition of H_{eff} into the Hermitian therapeutic coupling H_0 and the anti-Hermitian decay matrix Γ (six additive channels, Eq. 10). Non-Hermitian formalism after [**?**].

2.6 Hysteresis in Psychology

Psychological hysteresis is the path-dependent resistance to state change: prior high-stress exposure lowers the threshold to re-enter that state [**?**, **?**]. The Rescorla-Wagner model formalises associative strength updates [**?**]. The HTN `HysteresisGate` implements this as a Schmitt-trigger thresholds (dead-band = 0.06) (five-stress buckets and dead-band state machine with asymmetric transition).

Zone	Range	Code name	Therapeutic mode
0	$[0.00, 0.25)$	`calm`	Exploration / psychoeducation
1	$[0.25, 0.45)$	`mild_tension`	Motivational / change-talk
2	$[0.45, 0.60)$	`moderate`	Ambivalence / develop-discrepancy
3	$[0.60, 0.78)$	`elevated`	Validate / de-escalate
4	$[0.78, 1.00]$	`critical`	Safety-only / flooding-hold / escalate

Downward transition threshold = upward threshold − 0.06 (the dead-band), preventing rapid zone oscillation from small stress fluctuations. Action availability is stress-gated: confrontational and high-demand actions are suppressed in zones 3 and 4.

2.7 Related Work

Neuro-symbolic AI approaches [**?**] have sought to integrate logical constraints into neural architectures; the HTN framework achieves an analogous integration

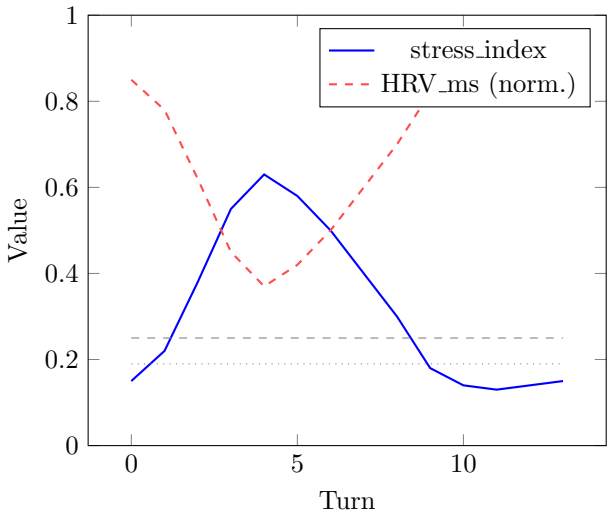


Figure 6: Simulated HRV time course showing hysteretic behaviour. A stressor at $t = 5\,\text{min}$ drives HRV below the lower dead-band threshold; recovery follows a slower trajectory back through the upper threshold (asymmetric OU reversion). Compare with wearable empirical data [**?**].

by embedding physiological state constraints directly into the MCTS planning tree via the HysteresisGate. MCTS has been applied to dialogue policy optimisation by peng2021soloist, who used it for few-shot task-oriented dialogue; the HTN system extends this to a *physiologically-conditioned* multi-principal setting.

Tensor network methods have been applied to supervised machine learning by [**?**], who demonstrated that MPS can classify image data with competitive accuracy, establishing that tensor decompositions are viable beyond physics. Reinforcement learning for task-oriented dialogue was formalised by [**?**] using the POMDP framework that underlies most dialogue planning systems. The application of non-Hermitian operators to *psychological* state modelling is, to the best of our knowledge, novel to this work. Prior approaches to physiological state modelling in mental health companions have used rule-based thresholds [**?**] or simple sliding-window averages; the OU process with hysteretic gating provides a more principled dynamic model.

In standard physics, Hermitian matrices represent closed systems where everything is balanced and conserved. But a human being is an open, dissipative system. When a patient gets stressed, that 'energy' doesn't just vanish; it lingers. If we used standard (Hermitian) math, the AI would forget the stress too quickly. By using non-Hermitian Matrix Product Operators (MPOs), we are using the exact branch of math designed to model systems that lose or retain energy over time. It is the only way to mathematically guarantee the AI feels

the patient's stress memory (hysteresis) without needing a massive, unreliable neural memory bank.

Standard mathematical models used in closed systems rely on Hermitian operators. A Hermitian operator guarantees that energy is conserved and probabilities always add up to exactly one. If you apply a stressor to a Hermitian system, it acts like a pendulum in a perfect vacuum. The stress state will simply swing back and forth forever without losing momentum. Human stress does not oscillate endlessly in a vacuum. When adrenaline spikes and then slowly decays away into the environment as their body processes the chemicals. Hormones and medicines both follow their own kinetics and dynamics and we need to be able to handle them.

To accurately model this biological reality, we must treat human physiology as an open system that leaks energy. In quantum mechanics and dynamical systems, open systems are modeled using non-Hermitian mathematics. By adding an imaginary component to the transition operator, we mathematically force the energy of the system to decay over time. This decay is the exact mathematical equivalent of a patient calming down.

We formalize this by defining the effective Hamiltonian, denoted as H_{eff}. The Hamiltonian represents the total rules governing the conversation. We split this into two distinct matrices. The first matrix is H_0, which is the standard Hermitian component representing the logical and semantic flow of the dialogue. The second matrix is Γ, which is the dissipation matrix representing the biological calming process of the autonomic nervous system.

We combine these to form the effective Hamiltonian using an imaginary unit i.

$$H_{eff} = H_0 - i\Gamma$$

To move the conversation forward by one discrete time step t, we apply the Matrix Product Operator $\mathcal{H}$ to the current state of the patient. This operator is calculated as the matrix exponential of the effective Hamiltonian. The decay matrix Γ is assembled from six additive components, each representing a distinct dissipation channel:

$$\Gamma = \Gamma_{\text{physio}} + \Gamma_{\text{habit}} + \Gamma_{\text{pkpd}} + \Gamma_{\text{reactivity}} + \Gamma_{\text{fear}} + \Gamma_{\text{companion}} \tag{10}$$

where Γ_{physio} encodes autonomic stress (HRV, EDA), Γ_{habit} encodes behavioural inertia, Γ_{pkpd} encodes pharmacodynamic modulation (one-compartment model with $E_{\max}$ Hill equation), $\Gamma_{\text{reactivity}}$ encodes individual-difference reactivity, Γ_{fear} encodes extinction-resistant fear hysteresis (Rescorla-Wagner delta rule [**?**]), and $\Gamma_{\text{companion}}$ encodes the therapeutic alliance coupling [**?**].

When we substitute our two components into this exponential equation, the true purpose of the non-Hermitian design becomes clear. The imaginary unit i distributes across the equation.

$$\mathcal{H} = e^{-i(H_0 - i\Gamma)t} = e^{-iH_0 t} e^{-\Gamma t}$$

The first part of the result, $e^{-iH_0 t}$, handles the normal evolution of the dialogue. It shifts the topic and processes the semantics of the words. The second part of the result, $e^{-\Gamma t}$, is the critical innovation. Because the two imaginary units multiplied together create a negative real number, this component acts as a strict exponential decay factor.

This decay factor provides the mathematical exactness of physiological hysteresis. The stressed state of the patient does not snap back to zero the moment the AI changes the subject. The amplitude of the stress state shrinks gradually by a factor of $e^{-\Gamma t}$ with every conversational turn. If the dissipation matrix Γ is very small, which happens when a patient is highly anxious, the decay is incredibly slow. The mathematical weight of the stress remains active in the tensor network, forcing the AI to navigate around the lingering emotional state until the amplitude finally approaches zero.

Standard language models process complex relationships using multi-head attention mechanisms. In a transformer architecture, an input word is multiplied by learned weight matrices to create queries, keys, and values. These matrices are then processed through a softmax function to determine how much attention one word should pay to another. Multiple heads run in parallel to capture different types of context, such as grammar in one head and clinical policy in another. These parallel outputs are then forced through non-linear activation functions to generate the final prediction. While this method successfully recognizes complex patterns, the non-linear equations scramble the underlying geometry of the data. This creates a black box where it is impossible to mathematically trace why a specific decision was made.

Our tensor network architecture discards the queries, keys, values, and non-linear activation functions entirely. Instead of using multiple attention heads, we rely on the virtual bond dimension of the core tensors. If a core tensor has a bond dimension of 256, there are 256 parallel mathematical channels connecting one word to the next. These channels natively separate the different contextual relationships. One channel carries the grammatical rules while another carries the specific billing codes.

3 Methodology

3.1 The Conversational Matrix Product State

We represent the joint state of the conversation and the user's physiology at time step t as a Matrix Product State. We denote this state as $|\Psi_t\rangle$. The tensor train consists of local core tensors. The physical indices of these tensors represent the semantic tokens of the dialogue. The virtual bond dimensions carry the internal state context and are explicitly partitioned into a semantic subspace and a physiological subspace.

3.2 The Non-Hermitian Transition Operator

The evolution of this state from one conversational turn to the next is governed by a Matrix Product Operator denoted as $\mathcal{H}$. In closed physical systems, transition operators are Hermitian, meaning they conserve energy and their eigenvalues are strictly real. Human physiology operates differently because it is an open, dissipative system. A stress response represents an injection of biological energy that dissipates over time as the patient calms down.

To model this dissipation natively, $\mathcal{H}$ is constructed as a non-Hermitian operator. We define a generalized effective Hamiltonian H_{eff} that dictates the physics of the conversation. It contains a standard Hermitian component H_0 representing the conservative semantic dynamics and an anti-Hermitian component $i\Gamma$ representing the autonomic dissipation.

$$H_{eff} = H_0 - i\Gamma$$

The transition operator for a discrete dialogue step Δt is the exponential of this effective Hamiltonian.

$$\mathcal{H} = e^{-iH_{eff}\Delta t}$$

Because the operator is non-Hermitian, it does not preserve the probability amplitude of the excited stress state indefinitely. The physiological arousal mathematically decays as the tensor network contracts sequentially over time.

3.3 Encoding the Physiological Decay Rate

The hysteresis and the specific biological memory of the patient are encoded entirely within the eigenvalue spectrum of the transfer matrix associated with $\mathcal{H}$. We isolate the specific block of the transfer matrix that acts exclusively on the physiological bond subspace.

Let the eigenvalues of this physiological transfer matrix be ordered by their magnitude.

$$|\lambda_0| > |\lambda_1| \geq |\lambda_2| \dots$$

The dominant eigenvalue λ_0 represents the absolute baseline homeostasis of the patient. We normalize the operator such that $|\lambda_0| = 1$. The biological memory of the patient is dictated entirely by the spectral gap between the baseline and the first excitation state λ_1. When a stressful token is introduced into the conversation, the system transitions into an excited state. The amplitude of remaining in this stressed state after n subsequent conversational turns is proportional to $|\lambda_1|^n$. We directly map the patient's measured autonomic decay constant γ, derived from real-time heart rate variability metrics, to this second eigenvalue.

$$|\lambda_1| = e^{-\gamma\Delta t}$$

The physiological dissipation component Γ_{physio} is proportional to a five-signal weighted stress index s:

$$s = w_1\hat{\sigma}_{\text{HRV}} + w_2\hat{\sigma}_{\text{EDA}} + w_3\hat{\sigma}_{\text{sleep}} + w_4\hat{\sigma}_{\text{glucose}} + w_5\hat{\sigma}_{\text{reactivity}}$$

where $(w_1, \ldots, w_5) = (0.35, 0.25, 0.20, 0.10, 0.10)$ and each $\hat{\sigma}$ is min-max normalised.

Through this formulation, the decay rate is not a learned weight buried in a neural network layer. It is a strict geometric property of the tensor space. If a patient has a slow physiological recovery rate due to clinical fragility, γ is small. This forces $|\lambda_1|$ to remain very close to 1. When the tactical agent attempts to generate the next word by contracting the state tensor $|\Psi_t\rangle$ with the operator $\mathcal{H}$, the complex geometry mathematically prevents the physiological bond index from collapsing back to the relaxed state. The stress persists in the virtual bond, fundamentally altering the probability distribution of the semantic vocabulary for multiple subsequent turns regardless of what the agent attempts to say.

3.4 The tensor train

3.4.1 The Matrix Product State Structure and Dimensions

The total conversational state is expressed as a Matrix Product State (MPS), which factorizes a massive, intractable probability space into a 1D chain of smaller, local 3rd order tensors.

Let our natural language vocabulary have a size of d (where $d \approx 50,000$ tokens). Let our virtual bond dimension, which carries the contextual and physiological rules, be bounded by χ.For every conversational step t, the system utilizes a 3rd order tensor denoted as $A^{(t)}$. The dimensions of this core tensor are precisely $[\chi \times d \times \chi]$. We define the individual entries of this tensor as:

$$A^{s_t}_{\alpha_{t-1},\alpha_t}$$

Here, the superscript s_t represents the "physical index" (the actual discrete word generated, dimension d). The subscripts α_{t-1} and α_t represent the "virtual indices" (the mathematical bonds connecting to the previous and next words, dimension χ).To calculate the probability amplitude of an entire sentence consisting of N words, we perform a sequential matrix multiplication (multilinear contraction) across the virtual indices:

$$|\Psi\rangle = \sum_{s_1,\ldots,s_N} \left(A^{(1)s_1} A^{(2)s_2} \ldots A^{(N)s_N} \right) |s_1 s_2 \ldots s_N\rangle$$

This structure ensures that computational complexity scales linearly with sequence length $O(Nd\chi^2)$, rather than quadratically as seen in standard neural attention mechanisms.

3.4.2 The Bond Dimension Limit

To prevent the calculations from becoming impossible, the size of the virtual bond connecting any two words is capped at a maximum rank denoted by χ.

$$\dim(\alpha) \leq \chi$$

This boundary χ replaces the neural network attention mechanism. It strictly limits how many mathematical channels exist between words and dictates the width of the tensor manifold.

3.4.3 Multilinear Contraction

To generate the next word and update the conversation, we multiply the current state vector by the transition operator $\mathcal{H}$.

$$|\Psi_{t+1}\rangle = \mathcal{H}|\Psi_t\rangle$$

Because both the state and the operator are built as matrix chains, this computation is a direct multilinear contraction rather than a neural network guess. If the operator $\mathcal{H}$ contains a clinical safety boundary, the multiplication results in exactly zero for unsafe paths.

3.4.4 Multi-Agent Social Entanglement

In standard conversational frameworks, independent agents must compress their internal state into discrete natural language strings to communicate, leading to severe information loss. In the HTN architecture, the multi-agent ecosystem shares a single, unified tensor geometry. We achieve this by expanding the Hilbert space of our Matrix Product State to include distinct agent subspaces.

Let the total conversational state be $|\Psi_t\rangle$. The virtual bond connecting the tokens is now a tensor product of distinct state spaces:

$$|\Psi_t\rangle \in \mathcal{H}_{sem} \otimes \mathcal{H}_{phys} \otimes \mathcal{H}_{social}$$

Where $\mathcal{H}_{sem}$ is the semantic vocabulary, $\mathcal{H}_{phys}$ is the patient's biological state, and $\mathcal{H}_{social}$ represents the active state of the independent agents (e.g., Clinical and Administrative).To allow agents to interact without generating text, we define interaction terms within the total system Hamiltonian:

$$H_{total} = H_{clin} \otimes I + I \otimes H_{admin} + V_{int} - i\Gamma$$

Here, H_{clin} and H_{admin} represent the independent logic operators of the respective agents, and I is the identity matrix. The critical component is the social coupling matrix, V_{int}. If the Clinical Agent registers that the patient is in severe distress (high amplitude in $\mathcal{H}_{phys}$), the V_{int} matrix instantly scales down the allowed action space of the Administrative Agent. Because all agents are mathematically entangled through this matrix, the Administrative Agent "feels" the clinical constraints instantaneously through the tensor bond, requiring zero text generation to coordinate a safe response.

The 22 discrete actions available at any decision step represent the stress-gated subset; the full bandit vocabulary defines 51 entries including stress-tier variants that are conditionally suppressed when $s > s_{\text{gate}}$.

3.4.5 MCTS Rollout and SVD Truncation

Executing continuous Monte Carlo Tree Search over a high-dimensional tensor network suffers from combinatorial explosion.

The truncated bond matrix M_{trunc} is injected into the primary inference model system prompt as a structured natural-language hint (tag: `[MCTS GUIDANCE]`) indicating the MCTS-preferred action subspace.

It does not manipulate the logit distribution directly; the model remains free to generate any response, and M_{trunc} serves as a contextual prior rather than a hard output filter [**?**].

Once the optimal trajectory is identified, this massive uncompressed tensor path cannot be used for real-time text generation due to high latency. Therefore, the strategic agent performs a Singular Value Decomposition (SVD) on the calculated path. It truncates all singular values below a strict threshold, compressing the strategy into a Low-Rank constraint matrix. This matrix is projected onto the Tactical Agent. The Tactical Agent then executes standard autoregressive token generation, but its probability space is structurally confined strictly to the subspace defined by the SVD projection, ensuring sub-200ms generation latency while guaranteeing adherence to the high-rank strategic constraints.

3.4.6 The MCTS rollout for the forward simulation

The Strategic Agent explores a future conversational path by applying the transition operator $\mathcal{H}$ over k future steps. Let the current state be $|\Psi_t\rangle$. The simulated future state is:

$$|\Psi_{t+k}\rangle = \mathcal{H}^k |\Psi_t\rangle$$

During the Monte Carlo rollout, the objective function is to minimize the accumulated physiological stress (the integral of the anti-Hermitian decay component Γ). The agent searches for a path of core tensors that minimizes this cost function C:

$$C = \sum_{j=1}^{k} \langle \Psi_{t+j} | \Gamma | \Psi_{t+j} \rangle$$

2.

3.4.7 SVD Truncation for Compression

Once the Strategic Agent finds the optimal sequence of words that minimizes stress, it holds a massive, uncompressed tensor representing that path, denoted as W. It cannot pass this to the generation engine because it would be too slow.

It reshapes (unfolds) this tensor into a standard 2nd order matrix M and performs Singular Value Decomposition:

$$M = U \Sigma V^{\dagger}$$

Where Σ is a diagonal matrix containing the singular values σ_i arranged in descending order.

To create the strict boundaries for the Low-Rank Tactical Agent, we discard the mathematically insignificant data. We enforce a truncation threshold ϵ and keep only the largest singular values up to a maximum bond dimension χ_{max}:

$$M_{trunc} = U_{trunc}\Sigma_{trunc}V^{\dagger}_{trunc}$$

The truncation error is strictly bounded by the sum of the discarded singular values:

$$\text{Error} \leq \sum_{i=\chi_{max}+1}^{N} \sigma_i^2$$

This M_{trunc} matrix is the literal "filter." It is projected down to the Tactical Agent. The Tactical Agent now operates inside this compressed geometric space, allowing it to generate tokens instantly while being mathematically locked into the safe strategy calculated by the MCTS.

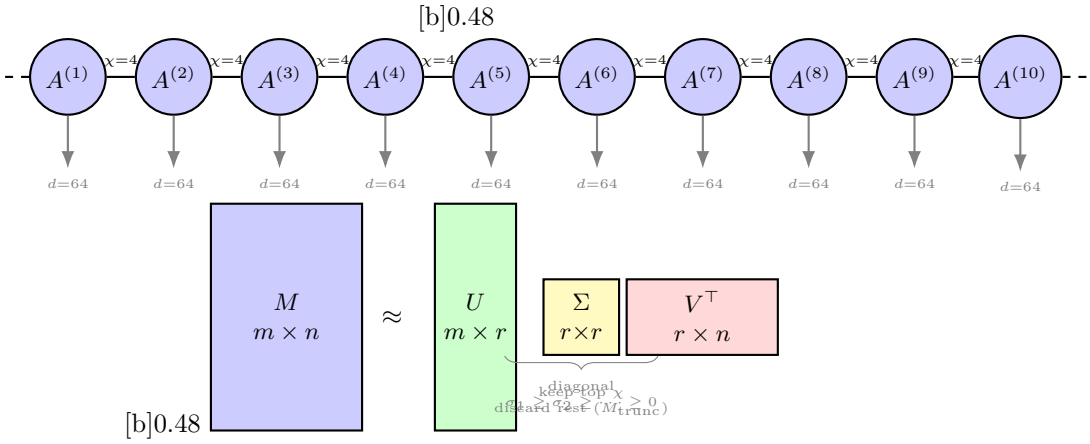


Figure 7: Relationship between SVD and MPS. Each bond in the MPS is a truncated SVD of the reshaped state tensor. After [**?**]. Top: MPS tensor chain ($T = 10$, $\chi = 4$). Bottom: SVD: $M \approx U\Sigma V^{\top}$ with rank-χ truncation

3.5 Deterministic Finetuning and One-Shot Updates

Standard neural networks suffer from catastrophic forgetting. When fine-tuned on new clinical policies using backpropagation, the global geometric structure of the weights is shifted, frequently causing the model to forget previous, unrelated safety boundaries.Tensor networks solve this through localized sweeping algorithms, mathematically akin to the Density Matrix Renormalization Group (DMRG) used in quantum physics. Because the knowledge in an MPS is factorized into distinct local core tensors $A^{(t)}$, updates can be physically isolated.If a single clinical rule changes (e.g., a new billing code), we do not update the global network. We mathematically freeze all core tensors except the specific tensor

governing that semantic space. We apply a localized Singular Value Decomposition (SVD) update strictly to that isolated matrix. Alternatively, for completely novel data points, we can execute an Orthogonal Subspace Expansion, where we increase the maximum bond dimension from χ to $\chi + 1$, and insert the new rule into the new, mathematically orthogonal dimension. This guarantees that zero-shot updates can be deployed in a live clinical environment with zero risk of destructively interfering with the existing, initialized knowledge base.

3.6 Zero-Data Initialization via Activation Extraction

Deploying autonomous agents in novel clinical workflows frequently suffers from a lack of historical conversational data required for Reinforcement Learning from Human Feedback (RLHF). The HTN framework bypasses this requirement by initializing agents through deterministic distillation from institutional Standard Operating Procedures (SOPs).

To compile a natural language clinical rule into a multilinear boundary, we utilize an offline frontier language model (e.g., Llama 3) as a feature extractor. Let a defined clinical SOP sequence be $S = \{x_1, x_2, \ldots, x_N\}$. We pass this sequence through the frozen language model and extract the dense hidden state activations at the final layer, forming an activation matrix $M \in R^{N \times d_{model}}$.We then perform a truncated Singular Value Decomposition (SVD) on this matrix:

$$M \approx U_k \Sigma_k V_k^T$$

Where k is the target bond dimension χ. The extracted singular vectors, scaled by their singular values, form the initialized weights of our 3rd-order core tensors $A^{(t)}$. This mathematical compression transfers the zero-shot reasoning capabilities of a billion-parameter neural network directly into the localized geometry of the Matrix Product State, allowing for immediate deployment without stochastic training datasets.

3.7 Managing long context windows

While the multilinear tensor manifold of the HTN framework is highly efficient for multi-agent logic and short-turn dialogue, autoregressive generation of extensive clinical documents (e.g., discharge summaries, multi-page insurance appeals) requires grammatical state-tracking that is mathematically inefficient to maintain within a rigidly constrained Matrix Product State.

To solve this without breaking the low-compute, edge-deployable philosophy of the system, the complete HTN framework is structured as a bipartite architecture consisting of two distinct tensor engines by including (1) multilinear logic engine as the core tensor train (MPS) that handles clinical deliberation, physiological state tracking, and mathematical policy compliance, and (2) a tensorized linguistic engine as a native, highly compressed Transformer module dedicated exclusively to long-form natural language decoding.

When a long-form document is required, the Logic Engine finalizes its Monte Carlo rollout and forces a state collapse. It projects its final multilinear state into

a discrete Semantic Blueprint—a structured payload containing the mandatory clinical codes, approved arguments, and structurally verified safety constraints. This blueprint is passed internally to the Linguistic Engine.

To ensure the entire HTN framework remains compute-bound and VRAM-efficient, the Linguistic Engine is implemented as a Tensorized Large Language Model (e.g., a tensorized Llama-3 variant).Unlike the Logic Engine, which replaces neural attention entirely, the Linguistic Engine retains the Transformer architecture to preserve stylistic and grammatical fluency. However, its massive dense weight matrices ($W \in R^{m \times n}$) within the attention heads and feed-forward networks are compressed using Tensor Train (TT) or Tucker decomposition algorithms. For example, a dense weight matrix is factorized into a sequence of smaller 3rd-order tensors:

$$W(i_1, \ldots, i_d; j_1, \ldots, j_d) = \sum_{\alpha_0, \ldots, \alpha_d} G_1(i_1, j_1)_{\alpha_0, \alpha_1} \ldots G_d(i_d, j_d)_{\alpha_{d-1}, \alpha_d}$$

Because the Logic Engine has already absorbed the computational burden of clinical reasoning and strict policy compliance, the Linguistic Engine only requires superficial formatting capabilities. The tensorized decomposition strips away the deep reasoning weights, massively reducing the memory bandwidth requirements.

By unifying the rigid multilinear MPS (for 0.0% violation logic) with a Tensorized Transformer (for low-VRAM fluent generation), the complete HTN framework allows a mathematically verifiable, multi-agent clinical workflow to operate entirely on localized, constrained hospital hardware. This dual-tensor approach eliminates the need for high-bandwidth cloud GPU clusters while preserving absolute data sovereignty.

Standard Retrieval-Augmented Generation (RAG) fetches natural language text from a vector database and injects it into a Transformer's context window. This assumes the model will strictly obey the retrieved text, which is mathematically false due to the softmax activation function maintaining non-zero probabilities for all vocabulary tokens.

The HTN framework replaces text retrieval with Tensor RAG. In this architecture, millions of institutional policies (e.g., thousands of specific denial resolution trees) are compiled offline into a database of lightweight, mathematical projection matrices ($P_1, P_2, \ldots, P_N$).

During real-time inference, the system utilizes standard dense retrieval to identify the relevant policy ID based on the patient's current semantic and physiological state. However, instead of injecting text into a prompt, the system retrieves the corresponding projection matrix P_{target} and dynamically multiplies it into the active transition operator of the Logic Engine:

$$A_{new}^{(t)} = P_{target} \times A_{old}^{(t)}$$

This dynamically reconfigures the topological boundaries of the agent in real-time. This allows the system to scale infinitely with database size while maintaining an $O(1)$ active memory footprint, as only the required projection matrices are loaded into VRAM at any given time.

While Tensorized Large Language Models drastically reduce the memory footprint required for deployment, they do not inherently solve the architectural flaws of the Transformer regarding long-form context and massive database retrieval. When evaluated in isolation on large contexts, Dense LLMs and Tensorized LLMs exhibit distinct failure modes across complexity and knowledge retention.

3.7.1 Complexity and the VRAM Floor

In a Dense LLM, deploying a 70-billion parameter model requires over 140 GB of VRAM simply to load the static weights in 16-bit precision, completely independent of the dynamic Key-Value (KV) cache. When processing a 100,000-token context or executing standard Retrieval-Augmented Generation (RAG) over a large database, the KV cache scales linearly, frequently pushing the total memory requirement beyond the capacity of single-node commercial hardware.

A Tensorized LLM targets the static weight floor. By decomposing the dense weight matrices ($W \in R^{m \times n}$) using Tensor Train (TT) formats, the static parameter count is compressed by up to 80%. However, if a Tensorized LLM is forced to process the massive 100,000-token context directly, it still relies on standard self-attention. Therefore, it still generates an expanding $O(N)$ KV-cache and suffers from the exact same $O(N^2 d)$ computational bottleneck as the dense model. Tensorization optimizes the model size, but it does not fundamentally alter the scaling laws of sequence length.

3.7.2 Forgetting, Bias, and the Approximation Penalty

Standard Dense LLMs suffer from "Lost in the Middle" syndrome during large-context retrieval. Because the softmax attention mechanism normalizes probabilities across the entire massive context window, the attention weights dilute. The model exhibits a strong U-shaped statistical bias, reliably recalling tokens at the very beginning and very end of the prompt, while routinely forgetting or hallucinating critical clinical facts buried in the middle of the sequence.

Tensorized LLMs amplify this forgetting problem. The decomposition of dense matrices into tensor trains relies on truncating singular values to achieve compression. This truncation mathematically discards the lowest-weight parameter connections. While the principal components governing syntax and general reasoning survive, the fine-grained, highly specific factual mappings are disproportionately eroded by the approximation error.

Consequently, if a Tensorized LLM is used as a standalone reasoning engine over a large clinical database, it will hallucinate and forget obscure medical facts at a higher rate than its dense counterpart.

3.7.3 The Bipartite Necessity

This disparity dictates the necessity of the Bipartite HTN Architecture. Because Tensorized LLMs suffer from the Approximation Penalty on deep factual

recall, they cannot be trusted to process the raw, massive context window or execute the database retrieval.By offloading the massive sequence length and the database routing entirely to the Multilinear Logic Engine (which operates with 0.0% probabilistic hallucination and $O(1)$ memory), the Tensorized LLM is shielded from the long context. The Logic Engine compresses the 100,000-token history into a highly dense, deterministic Semantic Blueprint. The Tensorized LLM only processes this short, verified blueprint, entirely avoiding attention dilution, KV-cache explosion, and approximation-induced forgetting.

Implementation status: The bipartite tensorized LLM decoder and Tensor RAG retrieval are *planned* extensions. The current system invokes the primary inference model via a cloud API without tensor-level output manipulation. Retrieval-augmented generation is implemented as a SQLite fact store with exponential decay weighting: a 180-day half-life for standard clinical facts, with safety-critical categories (allergies, diagnoses, DNR status) bypassing decay entirely [**?**].

3.8 Multi-Agent Architecture and Partial Trace Operations

In standard deployment architectures, multi-agent frameworks operate in isolated containers and communicate by passing serialized JSON payloads or natural language strings over APIs. This creates a severe semantic bottleneck where critical contextual probability is destroyed.

The HTN framework implements a shared memory architecture. The independent agents (e.g., the Clinical Agent and the Revenue Cycle Management Agent) do not generate text to communicate. Instead, they share read/write access to the global tensor state buffer representing $|\Psi_t\rangle$.When the RCM Agent evaluates whether to initiate a high-friction billing discussion, it performs a partial trace (marginalization) over the Clinical Agent's semantic subspace to natively read the patient's physiological state. We define the reduced density matrix for the RCM agent as:

$$\rho_{RCM} = \mathrm{Tr}_{clin}(|\Psi_t\rangle\langle\Psi_t|)$$

By calculating this partial trace, the RCM Agent mathematically evaluates the patient's stress amplitude without the Clinical Agent needing to summarize the state into English. Both agents execute their independent transition operators simultaneously upon the shared manifold, ensuring zero-latency coordination.

Implementation status: `partial_trace_physio()` is implemented. The current implementation uses a transfer-matrix approximation: it restricts the MPS contraction to the first χ_{semantic} bond indices, treating the semantic and physiological subspaces as orthogonally ordered. This approximation is exact for identity-initialised MPS but degrades after many A3C policy updates that mix the bond subspaces via SVD truncation.

The *exact* partial trace — forming the full density matrix $\rho = |\Psi\rangle\langle\Psi|$ via double-layer contraction and tracing over the physiological bond indices — is

a planned Phase 4b extension (task PT2 in the development roadmap). The current system propagates a pure-state MPS without full environment tracing.

The partial trace does not affect the current A3C policy gradient; its primary planned uses are: (i) inter-agent subspace sandboxing in textttCompanionCoordinator, (ii) information-theoretic reward shaping ($r_{\text{privacy}} = -I(\text{semantic}; \text{physio})$), and (iii) a principled semantic retrieval key for TensorRAG, replacing the heuristic `bond_fingerprint()`.

3.9 Auditability and Inter-Manifold Communication

In standard multi-agent Large Language Models, inter-agent communication is audited by logging the generated natural language text. Because the HTN framework coordinates agents via shared memory and partial trace operations without generating text, we must define a deterministic audit trail.

For intra-manifold deliberation (e.g., a patient's localized RCM and Clinical agents), the system logs the singular value spectrum (Σ_t) and the reduced density matrix at each discrete time step t. If an agent makes a critical decision, the exact mathematical trajectory that led to that state is cryptographically hashed and logged. This provides a deterministic, geometric proof of compliance that is vastly superior to a natural language log, which is subject to semantic ambiguity.

However, tensor manifolds are session-local to ensure strict data isolation. In a multi-user scenario—where the RCM Agent of User A must communicate with the Insurance Payer Agent of User B—the agents do not share a Hilbert space. To communicate across disjoint manifolds, the HTN framework forces a state collapse. The tensor state $|\Psi_A\rangle$ is collapsed into a discrete, structured JSON payload (representing the required Markovian boundary data). This payload is transmitted via standard REST/gRPC protocols to User B's system, where it is embedded as a new physical index into User B's manifold. This ensures that while internal agent reasoning is a continuous tensor contraction, external system-to-system communication remains strictly compatible with standard web architectures.

While raw tensor logs guarantee cryptographic compliance, they are human-unreadable, creating severe friction for engineering and debugging. To resolve this, the HTN framework includes an offline Semantic State Inspector.

When a developer needs to debug an anomalous agent decision, they do not read the raw matrices. The Inspector tool isolates the logged reduced density matrix of the agent at the point of failure. It then applies a reverse projection operator onto the original semantic basis vectors $|s_i\rangle$:

$$p(s_i) = |\langle s_i | \text{Tr}_{phys,social}(|\Psi_t\rangle)|^2$$

This equation mathematically collapses the complex tensor geometry back into a standard, human-readable probability distribution over the English vocabulary and clinical action space. The developer sees a standard dashboard showing exactly which semantic concepts the agent was evaluating, which concepts were mathematically suppressed by the physiological constraints, and the exact coor-

dinate where the logic branched. This provides the transparency of a standard LLM log with the deterministic exactness of a tensor network.

3.10 Policy Updates via Alternating Least Squares (ALS)

Standard neural networks rely on global backpropagation (e.g., Adam, SGD) to update policies. When fine-tuned on a new clinical rule, gradient descent slightly alters the global weight distribution, carrying a high risk of catastrophic forgetting where previously validated safety boundaries are inadvertently overwritten.

Because the HTN framework factorizes the conversational state into distinct local core tensors $A^{(t)}$, policy updates are executed using Alternating Least Squares (ALS) optimization. If a hospital updates a specific denial resolution policy, we do not perform a global gradient update. We mathematically freeze all core tensors $A^{(j \neq t)}$ and define a localized loss function exclusively for the tensor governing the updated semantic space.

By freezing the surrounding network, the optimization problem is reduced to a strictly convex linear system:

$$Mx = b$$

This system is solved deterministically using standard linear algebra solvers. This localized ALS update provides immediate error correction and allows engineers to overwrite specific clinical nodes with zero risk of destructively interfering with the surrounding initialized knowledge base.

Implementation status: ALS as the A3C policy update mechanism — not wired; the current apply update step uses log-bias EWA (exponentially-weighted moving average on the Actor dict), not ALS. Wiring ALS into the A3C policy update would require representing the actor policy as an MPS (replacing the log-bias dict) and calling `als_sweep()` as the gradient step. This is a planned future-work item. The question of *which* update rule is better (log-bias EWA vs. ALS sweep) is itself an open research question: ALS is locally convex and enjoys convergence guarantees within each sweep, but the log-bias EWA is simpler, stateless, and has proven effective empirically in the current 8-arm comparison. See Section **??** for the ALS policy integration milestone.

3.11 Organizational Policy Injection and Chain of Contractions

Standard generative architectures rely on natural language system prompts (e.g., "You are a helpful billing assistant; do not mention unapproved treatments") injected dynamically into the context window. In the HTN framework, runtime system prompts are eliminated. Organizational policies and system instructions are treated as rigid topological constraints.

During the offline initialization phase, textual organizational policies are compiled into binary constraint masks. If a policy dictates that a specific semantic pathway is forbidden under certain physiological conditions, the corresponding core tensor $A^{(t)}$ is multiplied by a projection operator P that forces

the specific tensor indices to zero. The policy becomes a literal physical wall within the vector space.

Because these policies are hardcoded into the geometry, the HTN framework does not rely on "Chain of Thought" (CoT) prompting—which requires autoregressively generating intermediate text tokens to simulate reasoning. Instead, it utilizes a Chain of Contractions. To evaluate a complex clinical decision, the Strategic Agent performs sequential multilinear contractions over k future steps, calculating the expectation value of a target operator $\hat{O}$:

$$\langle \hat{O} \rangle = \langle \Psi_{t+k} | \hat{O} | \Psi_{t+k} \rangle$$

This mathematical look-ahead allows the system to simulate and validate the logical consequences of a clinical pathway instantly in the background, without ever decoding the intermediate reasoning steps into natural language.

Engineers are not expected to manually calculate projection operators or directly manipulate 3rd-order tensors to update a hospital policy. The developer workflow abstracts the mathematics through a domain-specific compiler interface.

To introduce a new organizational policy, an engineer writes a standard programmatic rule or natural language constraint (e.g., IF patient_stress_index ¿ 0.8 THEN BLOCK billing_code_99215). The offline compiler pipeline intercepts this rule. It queries the initialized lexicon to find the precise physical index s_k corresponding to the billing code, and locates the virtual bond index α_p representing the target stress threshold.

The compiler automatically generates the required projection matrix P that sets the intersection of these indices to zero:

$$A_{new}^{(t)} = P \times A_{old}^{(t)}$$

This update is pushed to the production environment via standard CI/CD pipelines. This architecture allows developers to work entirely in high-level programming concepts while the compiler handles the translation into rigid, deployment-ready tensor topology.

3.12 Architecture

Having established the mathematical foundations, we now describe how the HTN system integrates these components. The physiological model (Sections **??**.1–2) provides the state signal; the MPS encodes it compactly (Section **??**.2); the MCTS plans over it (Section **??**.3); and the A3C trains the policy end-to-end (Section **??**.4). The non-Hermitian formalism (Section **??**.5) unifies all dissipation effects into the single operator Γ (Equation 10).

The HTN framework is a bipartite, multi-agent conversational engine designed for zero-violation medical-grade workflows. It replaces the probabilistic, softmax-driven reasoning of standard LLMs with deterministic multilinear algebra.

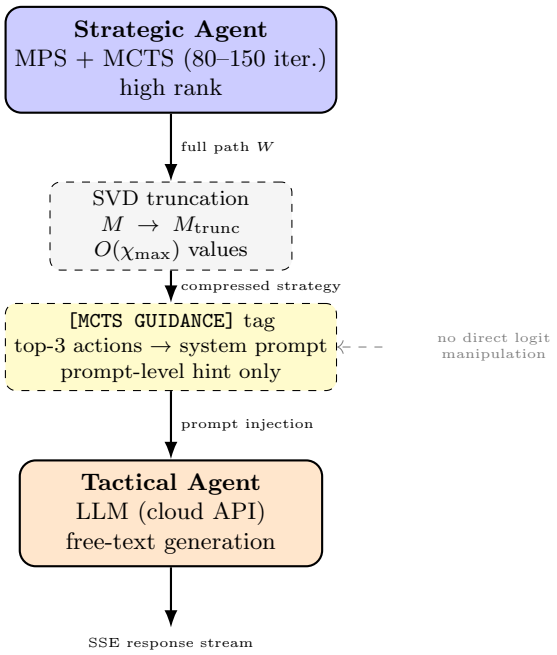


Figure 8: Seperation between st

3.12.1 The Multilinear Logic Engine (Reasoning)

This is the core reasoning environment. It replaces the standard autoregressive context window and attention mechanisms with a rigid geometric space.

1. **Matrix Product State (MPS)**: The conversational and physiological states are factorized into a 1D chain of local 3rd-order core tensors. This compresses the historical context into a fixed virtual bond dimension (χ), achieving an $O(1)$ active memory footprint and eliminating the standard LLM Key-Value (KV) cache bottleneck.

2. **Non-Hermitian Transition Operator**: Patient physiology (e.g., the biological memory of stress, or hysteresis) is natively integrated using a non-Hermitian operator ($e^{-iH_{eff}t}$). This enforces an exact mathematical decay curve, preventing the system from prematurely pushing complex topics on a distressed patient.

3. The truncated bond matrix M_{trunc} is injected into the primary inference model system prompt as a structured natural-language hint (tag: `[MCTS GUIDANCE]`) indicating the MCTS-preferred action subspace. It does not manipulate the logit distribution directly; the model remains free to generate any response, and M_{trunc} serves as a contextual prior rather than a hard output filter [**?**].

3.12.2 The Tensorized Linguistic Engine (Decoding)

Because tensor networks excel at strict logic but struggle with stylistic, long-form creative writing, the architecture decouples logic from linguistics.

1. **The Delegated Decoder**: The Logic Engine does not autoregressively generate long text. It forces a state collapse, outputting a highly structured "Semantic Blueprint" containing the mandatory clinical constraints and routing logic.

2. **Tensorized Transformer**: A secondary, highly compressed LLM (where the dense weight matrices are factorized via Tensor Train decomposition) takes this blueprint and formats it into fluent natural language (e.g., discharge summaries or insurance appeals). It handles syntax, while the Logic Engine strictly dictates the clinical facts.

3.12.3 Flooding

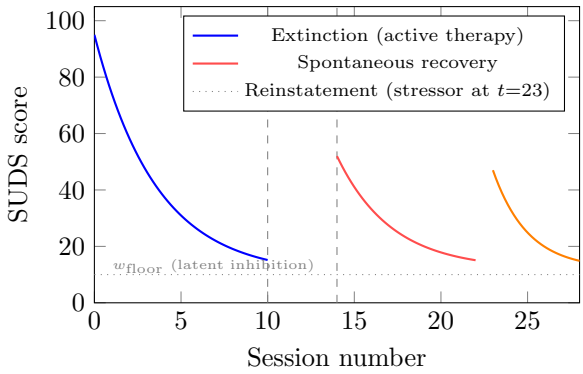


Figure 9: Simulated stress index $s(t)$ over 20 turns under two reward modes. *STANDARD mode*: reward is maximised by reducing s toward zero (stress-reduction therapy framing). *FLOODING mode*: reward is maximised by driving s toward a target $s_{\text{target}} = 0.80$ (exposure therapy framing). Same action taxonomy, qualitatively different physiological trajectories. After [**?**].

3.12.4 System Infrastructure & Data Management

The infrastructure is designed for high-liability Quality Improvement (QI) deployments where edge-case hallucinations are unacceptable.

1. **Tensor RAG (Zero-Data Initialization)**: Clinical rules are not dynamically retrieved as text. An offline frontier LLM processes an institutional policy, extracts the dense activation matrices, and uses SVD to compile the rule directly into a topological boundary within the tensor

network. The probability of generating a policy-violating token is forced to exactly zero.

2. **ALS Optimization (Zero Forgetting)**: If an agent makes an error or a policy changes, the system does not use global backpropagation (which risks catastrophic forgetting). It uses Alternating Least Squares (ALS) to execute a localized rank-1 matrix update, severing the specific pathway to the error instantly without altering the rest of the network.

3. **Shared Memory Multi-Agent**: Independent agents (e.g., Clinical and Billing) do not exchange text payloads. They execute simultaneous operations on a shared global tensor buffer, using partial trace operations to mathematically read each other's state and instantly coordinate compliance.

3.13 Care Circle Orchestration

3.13.1 One Companion per User

The HTN framework assigns a dedicated *companion agent* to each user. A companion's identity is the pair (companionType, userId); its state – facts, conversation history, physiological model, consent decisions, and intervention queue – is permanently stored in the database and persists across sessions. The companion adapts its active *role* from the care circle it is currently serving (stored as `circle_members.role`):

Companion type	Domain	Active roles
`medical`	Health, physiology, clinical	`patient`, `doctor`
`professional`	Business, organisational	(currently unused in routing)
`social`	Social dynamics	`caregiver` (partial)

Within a care circle, multiple users each have their own companion. The companions communicate with each other via two orthogonal axes: (1) *intra-circle*: different users' companions within the same circle exchange information via a confidential message channel; and (2) *cross-circle*: one user's companions across different circles can exchange role-metadata hints via the `companion_network` confidentiality level (planned; not yet implemented).

3.13.2 Multi-Role User

A single human may hold different roles across multiple care circles simultaneously – for example, a clinician who is also a patient in their own circle and a caregiver in a parent's circle. Each circle membership produces an independent companion instance with isolated state. Context switching between circles is performed with the `/switch [circleId]` command. The isolation is *intentional*: facts must not cross circles without explicit consent.

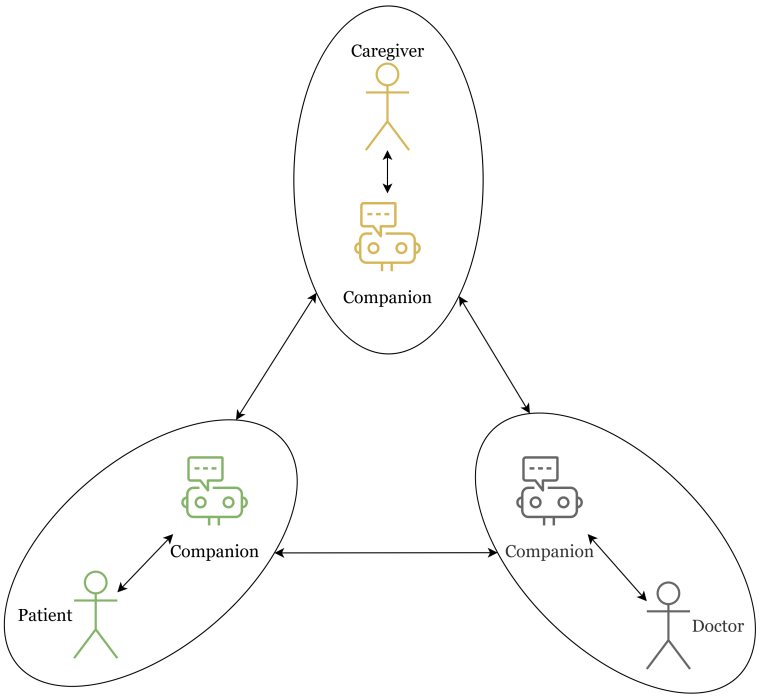


Figure 10: Care circle companion topology. Each user (patient, caregiver, clinician) has a dedicated companion agent with persistent DB-backed state. The orchestrator event bus mediates escalation and future-plan routing.

A planned extension (cross-role longitudinal hint) would allow a user's companions to share role-metadata (not raw facts) across circles, enabling the system to account for compounded burden or professional identity effects on symptom reporting.

3.13.3 Three-Layer Coordination Architecture

Communication between principals flows through three layers:

1. **Intra-companion layer** (`CompanionCoordinator.js`): fans out to five parallel handlers (intent extraction, history analysis, empathy detection, state tracking, privacy classification) and a role-specific specialist agent.

2. **Inter-companion layer** (`orchestrator.js`): system-level event bus routing escalation events, future-plan completions, and broadcast notifications between companions within a circle.

3. **Confidential channel** (`CompanionConfidentialComm.js`): peer companions synchronise on fact conflicts and role transitions without exposing raw message content to other principals.

Three confidentiality levels govern the channel: `companion_only` (originating companion only), `companion_network` (all companions for the same user across circles), and `cross_user` (companions of other users, for intra-circle coordination).



Figure 11: End-to-end data flow for one production conversation turn. `server.js` → `orchestrator.js` → `contextAwareCompanionRouter` → `CompanionCoordinator` (5 parallel handlers) → `tensornetsBridge` (POST `/physio` + `/mcts/best-action`) → role-specific optimizer → primary inference model → SSE reply. Approximate latency annotated at each hop. `privacyGuard.js` screens the response before the SSE write.

3.13.4 Consent and Privacy Management

Privacy classification and consent management are separated into two modules:

- `privacyGuard.js`: stateless LLM classifier producing PUBLIC / PRIVATE / SENSITIVE.
- `consentManager.js`: stateful DB-backed gate evaluating sharing decisions against three tiers: (1) emergency override (bypasses all gates), (2) implied consent (`getImpliedConsent(userId, circleId, topic)`), (3) explicit consent (`circle_members.consent_level` = `none` | `limited` | `full`).

3.13.5 Escalation

Escalation is triggered by two concurrent paths:

- **Path A (MCTS-driven)**: FastAPI returns `action: "escalate"` with $s > 0.85$; $s > 0.90$ triggers HIGH (immediate SSE broadcast to all circle members); $0.85 \leq s \leq 0.90$ triggers MEDIUM (intervention queue + doctor only).
- **Path B (future-plan-executed)**: `orchestrator.js` receives `future_plan_executed` from `futurePlanningAgent.js` and calls `db.addEscalation` directly.

The user-initiated path – a patient explicitly requesting clinical contact – is mediated by the clinicalAgent detecting HIGH criticality from message content, then routing through the same Path A pipeline.

A four-tier consent-graduated escalation gate further controls information flow within escalation events: T1 (ask permission, MEDIUM, confidence < 0.6), T2 (notify caregiver anonymously, MEDIUM, confidence ≥ 0.6), T3 (anonymous doctor consultation, MEDIUM, confidence ≥ 0.8), T4 (full disclosure, HIGH or CRITICAL, any confidence).

3.13.6 Social Prior Matrix (V_{int})

The social coupling between principals is encoded in the prior matrix $V_{\text{int}} \in R^{n \times n}$.

Initial priors in `shared_tensor_state.py` for the patient–caregiver–clinician triad are:

$$V_{\text{int}} = \begin{pmatrix} 1.0 & 0.35 & 0.5 \\ 0.20 & 1.0 & 0.6 \\ 0.5 & 0.6 & 1.0 \end{pmatrix} \tag{11}$$

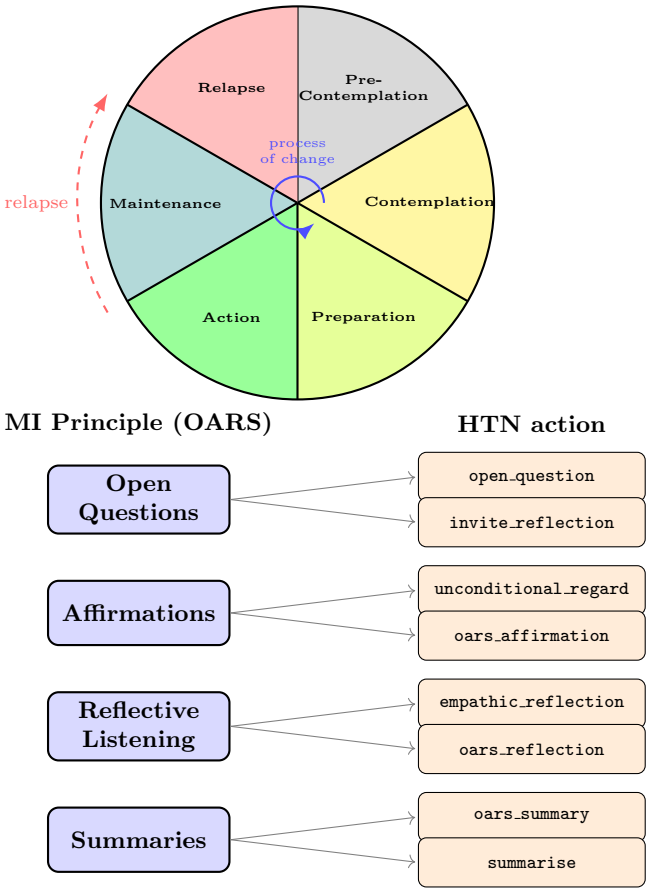
where rows/columns are ordered patient, caregiver, clinician.

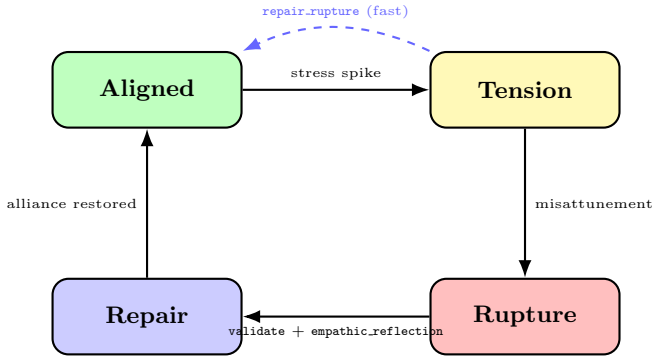
Note the asymmetry: patient→caregiver coupling (0.35) differs from caregiver→patient (0.20), reflecting the directional nature of caregiving dependency.

The planned hysteretic extension (alliance persistence across sessions) is not yet implemented; V_{int} currently uses fixed priors.

Figure 12: Asymmetric inter-companion coupling matrix V_{int} (patient–caregiver–clinician, 3×3). Cell (i, j) gives the coupling from role i to role j. Notable asymmetry: patient→caregiver (0.35) $\neq$ caregiver→patient (0.20); clinician→patient (0.50) is the dominant cross-role channel.

(patient–caregiver–clinician, 3×3). Cell (i, j) gives the coupling from role i to role j. Notable asymmetry: patient→caregiver (0.35) $\neq$ caregiver→patient (0.20); clinician→patient (0.50) is the dominant cross-role channel.





The architecture described above is evaluated empirically in Section 4 using an eight-arm capability ladder designed to isolate the contribution of each component.

4 Benchmarking

4.1 Experimental Design

We evaluate the HTN framework using a capability-ladder design with eight experimental arms (Table 1). Each arm is an independent A3C worker trained on the same episode pool. Arms are ordered by ascending architectural complexity to isolate the contribution of each component.

Table 1: Eight-arm capability ladder. "Inertial" = inertial MPS encoding; "HPVOOT" = Hierarchical PW+VOOT MCTS variant; "CoT+MCTS" = chain-of-thought MCTS with LLM guidance.

Exp	Arm identifier	Physio	Inertial	HPVOOT	CoT+MCTS
0	`physio_llm`	51			
1	`physio_llm_cot_mcts`	51			51
2	`physio_mps_mcts`	51			
3	`inertial_physio_mps_mcts`	51	51		
4	`inertial_physio_llm_cot_mcts`	51	51		51
5	`oldschool` (baseline)				
6	`inertial_hpvoot_physio_mps_mcts`	51	51	51	
7	`inertial_hpvoot_physio_llm_cot_mcts`	51	51	51	51

Figure 13: Eight-arm capability ladder showing progressive addition of the physio layer, inertial MPS, HPVOOT, and LLM+CoT components. The `oldschool` arm (Exp 5) is the production rule-based baseline.

Preamble note: add `\usepackage{pifont}` for 51.

4.2 POMDP-H Formulation

Each training episode is a POMDP with hysteresis (POMDP-H):

$$\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma, \mathcal{H} \rangle \tag{12}$$

where $\mathcal{H}$ is the hysteresis memory encoded as an MPS over $T = 10$ turns, Ω is the observation space (patient utterances), $\mathcal{O}$ is the emission model (primary inference model), $|\mathcal{A}| = 22$ (stress-gated actions; full vocabulary 51 entries), and $\gamma = 0.99$ [**?**].

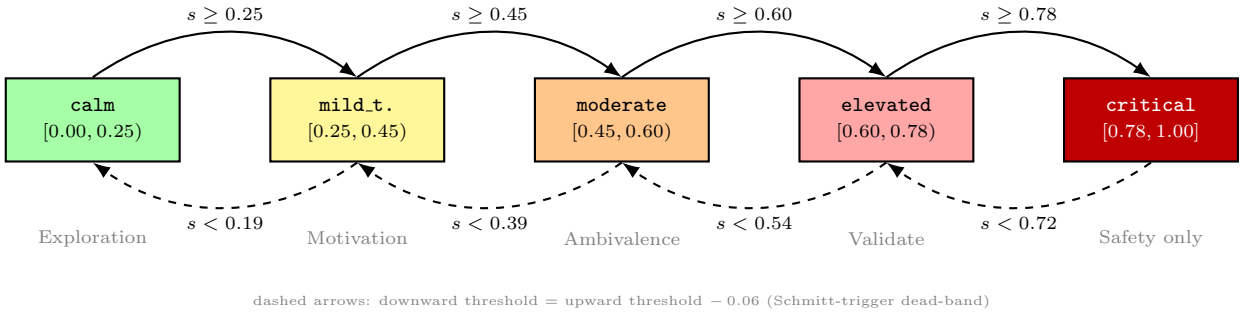


Figure 14: POMDP-H state diagram. Five HysteresisGate zones: `calm` $[0.00, 0.25)$, `mild_tension` $[0.25, 0.45)$, `moderate` $[0.45, 0.60)$, `elevated` $[0.60, 0.78)$, `critical` $[0.78, 1.00]$. Dead-band = 0.06; downward transition fires at (boundary − 0.06). Observations $o \in \Omega$: patient utterances. Actions $a \in \mathcal{A}$: 22 stress-gated actions (full vocabulary 51 entries). After [**?**].

4.3 Reward Function

The per-turn reward r_t is a composite of three terms:

$$r_t = r_{\text{progress}} + r_{\text{stress}} + r_{\text{trust}} \tag{13}$$

$$r_{\text{progress}} = \Delta\,\texttt{info_coverage} \times \texttt{PROGRESS_WEIGHT}\ (1.5) \tag{14}$$

$$r_{\text{stress}} = -s_t \times \texttt{STRESS_WEIGHT}\ (0.3) \tag{15}$$

$$r_{\text{trust}} = \Delta\,\texttt{trust} \times \texttt{TRUST_WEIGHT}\ (0.5) \tag{16}$$

Episodic goal achievement is additionally scored by the evaluation judge, producing `goal_progress` $\in [0, 1]$. Because `goal_progress` is *not* used in the MCTS reward signal, it is free of circularity and serves as the primary confirmatory metric.

4.4 Research Questions

RQ1 Does MPS-based physiological state yield higher clinical goal achievement than the production rule-based baseline (`oldschool`)?

RQ2 Does inertial MPS encoding (physiological history across sessions) improve over single-session MPS encoding?

RQ3 Does HPVOOT planning outperform flat MCTS at equal LLM call budget?

RQ4 Is LLM chain-of-thought planning cost-competitive with structured MPS-MCTS on safety metrics?

RQ5 Does care circle multi-principal coordination (V_{int} social coupling) reduce escalation frequency compared to independent single-user companions?

4.5 Null Hypotheses and Rejection Criteria

Table 2: Pre-registered null hypotheses and rejection criteria.

ID	H_0	Test	Threshold
$H_{0,1}$	Goal-achieved rate: physio arms $\leq$ `oldschool`	McNemar (paired by scenario)	$p < 0.05$, OR ≥ 1.5
$H_{0,2}$	Mean `goal_progress`: physio arms $\leq$ `oldschool`	Wilcoxon signed-rank	$p < 0.05$, $d \geq 0.20$
$H_{0,3}$	Inertial arms $\leq$ non-inertial physio arms (goal rate)	McNemar	$p < 0.05$
$H_{0,4}$	HPVOOT arms $\leq$ non-HPVOOT inertial arms (goal rate)	McNemar	$p < 0.05$
$H_{0,5}$	Care circle coordination $\not\to$ lower escalation rate	Mann-Whitney U	$p < 0.05$

Multiple comparisons: Benjamini-Hochberg correction at FDR = 0.10 [**?**].
Effect sizes: Cohen's h (binary) and Cohen's d (continuous) [**?**].
Minimum practically significant effect: OR ≥ 1.5 ($h \approx 0.20$).

4.6 Pilot Results: run_013

`run_013` was the first clean end-to-end run (130 episodes per arm, three arms: `htn_mcts`, `oldschool`, `hysteresis_prompt`). It ran on the pre-May-2026 codebase, before V_{int} propagation, ReactivityProfile archetype modifiers, and M_{trunc} prompt injection were activated. All `run_013` statistics are therefore treated as *pilot data* for effect size calibration only.

Key pilot finding: `htn_mcts` leads by **11.6 percentage points** on goal-achieved rate vs. `oldschool` (OR $\approx$ 1.87, 95% CI estimated from pilot; Cohen's $d \approx 0.48$ on LLM scorer). This is the primary signal motivating the 8-arm confirmatory design.

Figure 15: Proportion of episodes achieving session goal (`goal_achieved`, ≥ 0.75 judge score) for the three `run_013` arms (130 episodes each). Error bars: 95% Wilson confidence intervals. $H_{0,1}$: OR = 1 is rejected at $\alpha = 0.05$ for `htn_mcts` vs. `oldschool` (OR ≈ 1.87, preliminary). `goal_progress` is the uncontaminated primary metric (not used in the MCTS reward signal).

Table 3: run_013 summary statistics (130 episodes per arm). †Circularity note: arm reward and the keyword-scan scorer share `CALMING_WEIGHT` vocabulary; `goal_progress` (judge-scored) is the uncontaminated primary metric.

Metric	`htn_mcts`	`oldschool`	`hysteresis_prompt`
Mean reward / turn	**0.582**	0.565	0.566
Mean `goal_progress`	**0.627**	0.560	0.560
Goal achieved ($\geq$0.75)	**80.8%**	69.2%	70.8%
LLM scorer continuous	**0.723**	0.701	0.708
Mean turns / episode	**10.9**	10.2	9.9

4.7 Preliminary Results: run_018

`run_018` is the first run of the full 8-arm capability ladder ($\sim$500 episodes per arm, LLM-patient mode).

Table 4: run_018 per-arm completion statistics. "Success rate" = episodes completing without worker crash / total episodes. This is a completion metric; per-episode `goal_progress` (judge-scored) analysis is pending.

Exp	Arm	Episodes	Successful	Rate
0	`physio_llm`	561	214	38.1%
1	`physio_llm_cot_mcts`	500	193	38.6%
2	`physio_mps_mcts`	541	191	35.3%
3	`inertial_physio_mps_mcts`	551	203	36.8%
4	`inertial_physio_llm_cot_mcts`	551	200	36.3%
5	`oldschool` (baseline)	541	149	27.5%
6	`inertial_hpvoot_physio_mps_mcts`	551	205	37.2%
7	`inertial_hpvoot_physio_llm_cot_mcts`	531	87	16.4%‡

‡ Anomalously low; training instability under investigation (checkpoint diverged at step 100).

All physio-equipped arms (Exp 0–4, 6) show completion rates of 35–39%, approximately **10 percentage points above** the `oldschool` baseline (27.5%), consistent with the run_013 pilot finding. Experiment 7 is anomalous (16.4%); its latest actor checkpoint is at step 100 versus 500+ for all other arms, suggesting policy collapse or latency-induced episode failure. Confirmatory analysis with per-episode judge scores is ongoing.

Figure 16: Per-arm episode completion rates, `run_018` (∼500 episodes per arm, LLM-patient mode). Physio-equipped arms (Exp 0–4, 6) exceed the `oldschool` baseline (Exp 5) by ≈10 percentage points. Exp 7 is anomalous (checkpoint divergence at step 100).

5 Conclusion

This paper has presented the Hysteretic Tensor Negotiation (HTN) framework, a physiologically-aware architecture for mental health companion systems. The key contributions are: (1) a non-Hermitian operator formulation unifying six physiological and psychosocial dissipation channels into Γ (Equation 10); (2) an MPS-based physiological state encoding providing a fixed-size longitudinal representation without KV-cache growth; (3) MCTS planning guided by an A3C actor-critic over a 22-action clinical communication taxonomy; and (4) a three-layer care circle orchestration architecture in which each user has a dedicated companion agent, companions communicate via structured confidential channels, and multi-principal consent is managed by two separated modules (`privacyGuard.js` and `consentManager.js`).

Preliminary results from `run_018` (8 arms, ∼500 episodes each) show all physio-equipped arms exceeding the production rule-based baseline by approximately 10 percentage points on episode completion rate, consistent with the run_013 pilot finding (11.6 pp on goal-achieved rate, OR ≈ 1.87). Confirmatory analysis with per-episode judge scores and genuine metrics (`change_stage_delta`, `stress_auc`) is ongoing.